



DOCUMENTATION CODE

Commentaires & Architecture du Code

schwartz_smith_oil.py — Python 3.12 — Kalman-MLE

GUIDE COMPLET LIGNE PAR LIGNE

Février 2026

1. ARCHITECTURE GÉNÉRALE DU CODE

1.1 Vue d'ensemble

Le fichier `schwartz_smith_oil.py` est structuré en 10 fonctions pures organisées dans un pipeline séquentiel. Cette organisation permet de suivre le pipeline étape par étape, et de tester chaque fonction séparément si besoin.

Fonction	Lignes	Rôle
<code>load_and_clean()</code>	~25	Chargement Excel, décodage des symboles, calcul maturités
<code>build_piliers()</code>	~35	Interpolation sur piliers fixes 30/90/180/360 jours
<code>schwartz_smith_kalman()</code>	~70	Filtre de Kalman complet , cœur mathématique
<code>neg_log_likelihood()</code>	~15	Fonction objective pour l'optimiseur MLE
<code>calibrate ()</code>	~30	Lancement L-BFGS-B, affichage des paramètres
<code>extract_states()</code>	~20	Post-traitement des états, calcul demi-vie
<code>monte_carlo()</code>	~25	Simulation 500 trajectoires, percentiles P10/P50/P90
<code>stress_test()</code>	~20	Choc géopolitique avec κ réduit et vol amplifiée
<code>fig_*() × 6</code>	~200	Génération des 6 dashboards Plotly interactifs
<code>main()</code>	~40	Orchestration du pipeline, sauvegarde, export CSV

1.2 Flux de données

```

Db_CL.xlsx
└─ load_and_clean() → df_base (DataFrame brut + maturités)
    └─ build_piliers() → prices_mat (matrice T×4 log-prix)
        └─ calibrate() → params (vecteur 8 paramètres)
            └─ extract_states() → states, spot, long_term
                └─ monte_carlo() → sim_paths,
                    P10/P50/P90
                        └─ stress_test() → stress_paths
                            └─ fig_*() → HTML
                                └─ export_powerbi() →
                                    CSV

```

2. CHARGEMENT ET PRÉPARATION DES DONNÉES

2.1 Fonction `load_and_clean()`

C'est la première fonction appelée. Elle charge les données et fait quelques vérifications de base avant de passer à la suite.

```
def load_and_clean(filepath: str) -> pd.DataFrame:
```

```
df = pd.read_excel(filepath)
df['Date'] = pd.to_datetime(df['Date'])
df = df.sort_values('Date').reset_index(drop=True)
```

💡 Le tri par date est critique : le filtre de Kalman suppose des observations chronologiquement ordonnées. Sans ce tri, les matrices de transition seraient incorrectes.

Décodage des symboles futures

Les contrats WTI sont codés selon la convention CME standard : CLG00 = Crude Light, G = février, 00 = 2000. La table MONTH_CODES est le dictionnaire de décodage :

```
MONTH_CODES = {
    "F": 1, "G": 2, "H": 3, "J": 4, "K": 5, "M": 6,
    "N": 7, "Q": 8, "U": 9, "V": 10, "X": 11, "Z": 12
}

# Exemple : symbole 'CLH18'
code = 'CLH18'[-3:] # → 'H18'
m_code = 'H18'[0] # → 'H' = mars
y_code = '18' # → 2018
mois = MONTH_CODES['H'] # → 3
annee = 2000 + 18 # → 2018
```

Calcul des maturités résiduelles

Mat_jours est la distance en jours entre la date d'observation et la date d'expiration du contrat. C'est la variable clé τ (tau) du modèle Schwartz-Smith.

```
df['Maturity'] = pd.to_datetime({
    'year': years, 'month': months, 'day': 1}, errors='coerce')
df['Mat_jours'] = (df['Maturity'] - df['Date']).dt.days
```

3. CONSTRUCTION DE LA STRUCTURE DE TERME

3.1 Fonction build_piliers ()

Le filtre de Kalman a besoin d'une matrice de taille fixe. Problème : les données de marché ont des maturités qui changent chaque jour. build_piliers() contourne ça en interpolant les prix sur 3 maturités fixes.

Paramètre global PILIERS_JOURS

```
PILIERS_JOURS = np.array([20, 40, 60])
PILIERS_ANS = PILIERS_JOURS / 365.25 # conversion en années pour les formules
```

⚠ Ce choix de piliers (20/40/60j) est crucial. Les piliers initiaux étaient trop proches le modèle ne voyait pas la dynamique de retour à la moyenne et produisait $\kappa \rightarrow 0$ (demi-vie de 843 jours). Les piliers longs fournissent l'information fondamentale sur ξ .

Algorithme d'interpolation par fenêtre glissante

```
for i in range(len(df)):
    # Fenêtre ±2 jours autour de l'observation i
    window = df.iloc[max(0, i-2) : min(len(df), i+3)]

    # Suppression des doublons d'abscisse (même maturité, prix différents)
    _, unique_idx = np.unique(x_win, return_index=True)
    x_win, y_win = x_win[unique_idx], y_win[unique_idx]

    # Interpolation linéaire + extrapolation si nécessaire
    f = interp1d(x_win, y_win, kind='linear', fill_value='extrapolate')

    # Garde-fou : clamp entre log(5$) et log(200$)
    piliers_vals = np.clip(f(PILIERES JOURS), np.log(5), np.log(200))
```

💡 Les données sont en log-prix (ln du prix en \$) car le modèle Schwartz-Smith opère dans l'espace logarithmique. `np.clip(... log(5), log(200))` élimine les artefacts d'interpolation hors de la plage physiquement plausible pour le WTI.

4. LE FILTRE DE KALMAN , CŒUR MATHÉMATIQUE

4.1 Fonction `schwartz_smith_kalman()`

C'est clairement la fonction centrale. Elle fait tourner le filtre de Kalman pour le modèle de Schwartz & Smith. En pratique elle est appelée des milliers de fois pendant la calibration, puis une dernière fois pour extraire les états finaux.

Initialisation des matrices

```
# Vecteur d'état initial : chi=0 (pas de choc), xi=prix observé initial
x = np.array([0.0, prices_mat[0, 0]])
P = np.eye(2) * 0.5 # Incertitude initiale modérée

# Matrice de transition : évolution de [chi, xi] sur un pas dt = 1/252
# chi(t+dt) = exp(-kappa*dt) * chi(t) [retour exponentiel à 0]
# xi(t+dt) = 1 * xi(t) [marche aléatoire pure]
G = np.array([[exp(-kappa * dt), 0.0],
              [0.0, 1.0]])
```

💡 $\exp(-\kappa \cdot dt)$ est le facteur de décroissance exponentielle de χ . Pour $\kappa=2.5$ et $dt=1/252$: $\exp(-2.5/252) \approx 0.9901$. χ perd ~1% de sa valeur chaque jour de trading cohérent avec une demi-vie de ~100 jours.

Matrice de covariance du bruit de processus W

W est la partie qui m'a donné le plus de mal. Elle doit être semi-définie positive, sinon le filtre explose numériquement. J'ai ajouté une correction automatique pour éviter ça :

```
var_chi = (s_chi**2 / (2*kappa)) * (1 - exp(-2*kappa*dt)) # Variance chi
var_xi = s_xi**2 * dt # Variance xi
cov_term = rho*s_chi*s_xi/kappa * (1 - exp(-kappa*dt)) # Covariance

W = np.array([[var_chi, cov_term],
              [cov_term, var_xi ]])

# Vérification SDP : correction si valeur propre minimale < 0
W = (W + W.T) / 2 # Symétrisation numérique
min_eig = np.linalg.eigvalsh(W).min()
if min_eig < 0:
    W += (-min_eig + 1e-9) * np.eye(2) # Correction epsilon
```

⚠ Sans la vérification SDP, certaines combinaisons de paramètres (ρ proche de ± 1 , κ très faible) peuvent rendre W indéfinie, causant des erreurs numériques lors de la décomposition de Cholesky implicite dans le filtre.

Matrice d'observation H et constante A(τ)

H fait le lien entre les états cachés et les prix qu'on observe. Chaque ligne correspond à une maturité τ différente :

```
H[i, 0] = exp(-kappa * tau) # Coefficient de chi : décroît avec la
maturité
H[i, 1] = 1.0 # Coefficient de xi : constant (impact
permanent)

# A(tau) : terme de convexité complet (Schwartz & Smith 2000, eq.10)
# Inclut primes de risque, corrections d'Itô, termes de corrélation
A[i] = (
    -(l_chi/kappa) * (1 - exp(-kappa*tau)) # Prime risque CT
    + (mu_xi - l_xi - 0.5*s_xi**2) * tau # Drift LT ajusté
    + (s_chi**2/(4*kappa)) * (1 - exp(-2*kappa*tau)) # Convexité chi
    + (s_xi**2/2) * tau # Convexité xi
    + (rho*s_chi*s_xi/kappa) * (1 - exp(-kappa*tau)) # Corrélation
)
```

💡 La constante A(τ) est nulle dans une version simplifiée du modèle. Notre implémentation inclut les 5 termes de l'équation 10 de Schwartz & Smith (2000) cela corrige le biais systématique sur les prix forward des maturités longues.

Étapes prédiction et mise à jour (Kalman classique)

```
# — PRÉDICTION —————
x_pred = G @ x + C # État prédit
P_pred = G @ P @ G.T + W # Covariance prédite

# — INNOVATION —————
innov = z - (H @ x_pred + A) # Erreur de prédiction
S = H @ P_pred @ H.T + R # Variance de l'innovation

# — LOG-VRAISEMBLANCE (contribution de t) —————
sign, logdet = np.linalg.slogdet(S)
```

```
log_lik -= 0.5 * (N*log(2π) + logdet + innov @ solve(S, innov))

# — MISE À JOUR —————
K = P_pred @ H.T @ inv(S)          # Gain de Kalman
x = x_pred + K @ innov              # État mis à jour
P = (eye(2) - K @ H) @ P_pred      # Covariance mise à jour
```

💡 Le gain de Kalman K est le poids optimal accordé à l'innovation (erreur de marché). Si K est grand : on fait confiance aux données de marché. Si K est petit : on fait confiance au modèle. C'est l'équilibre automatique au sens des moindres carrés.

5. CALIBRATION PAR MAXIMUM DE VRAISEMBLANCE

5.1 Fonction neg_log_likelihood()

Cette fonction sert d'intermédiaire entre le filtre de Kalman et scipy. Elle retourne l'opposé de la log-vraisemblance (puisque'on minimise), avec quelques garde-fous pour éviter les plantages numériques.

```
def neg_log_likelihood(params, prices_mat, taus, dt):
    kappa, s_chi = params[0], params[1]

    # Garde-fous : rejet des paramètres physiquement impossibles
    if kappa < 0.8 or kappa > 5.0 or s_chi > 0.6:
        return 1e15 # Valeur pénalisante = ce paramètre est rejeté

    try:
        _, ll = schwartz_smith_kalman(params, prices_mat, taus, dt)
        return -ll if np.isfinite(ll) else 1e15
    except Exception:
        return 1e15 # Tout échec numérique → rejet
```

5.2 Fonction calibrate()

Point de départ (initial guess)

```
# kappa=2.5 → demi-vie ~101 jours (centre de la plage cible)
x0 = [2.5, 0.25, 0.15, 0.30, 0.04, 0.0, 0.0, 0.04]
#      κ   σ_χ   σ_ξ   ρ   μ_ξ   λ_χ   λ_ξ   σ_ε
```

Bornes d'optimisation

Paramètre	Borne min	Borne max	Justification
κ (kappa)	0.8	5.0	Demi-vie entre 51 et 315 jours
σ_χ	0.01	0.60	Vol CT entre 1% et 60% annualisée
σ_ξ	0.01	0.40	Vol LT bornée stabilité du fondamental
ρ	-0.9	+0.9	Corrélation strictement inférieure à 1
μ_ξ	0.00	0.12	Dérive LT positive et modérée

Paramètre	Borne min	Borne max	Justification
$\lambda_{\chi}, \lambda_{\xi}$	-0.2	+0.2	Primes de risque faibles (cohérent CAPM)
σ_{ε}	0.005	0.15	Bruit de mesure entre 0.5% et 15%

💡 L'optimiseur L-BFGS-B (Limited-memory Broyden-Fletcher-Goldfarb-Shanno Bounded) est le standard industriel pour les problèmes d'optimisation contrainte à gradient. Il converge typiquement en 100–300 itérations sur cette calibration.

6. SIMULATION DE MONTE CARLO

6.1 Fonction monte_carlo()

On repart des derniers états du filtre de Kalman et on simule 500 trajectoires sur un an de trading (252 jours ouvrés).

```
for s in range(n_sims):          # 500 simulations indépendantes
    chi, xi = chi_0, xi_0        # Conditions initiales = état Kalman
    final
    for t in range(horizon):      # 252 pas de temps quotidiens
        # Deux browniens indépendants (corrélation non simulée ici)
        dW1, dW2 = np.random.normal(0, np.sqrt(dt), 2)

        # Évolution chi : Ornstein-Uhlenbeck discret
        chi += -kappa * chi * dt + s_chi * dW1

        # Évolution xi : GBM avec dérive
        xi += (mu_xi - 0.5*s_xi**2) * dt + s_xi * dW2

        # Prix spot = exp(chi + xi) [retour à l'espace des prix]
        paths[t, s] = np.exp(chi + xi)
```

⚠️ Note technique : la corrélation ρ entre $dW1$ et $dW2$ n'est pas simulée explicitement ici ($dW1$ et $dW2$ sont indépendants). Pour une simulation rigoureuse, il faudrait utiliser une décomposition de Cholesky : $dW = L @ dZ$ où $L = \text{chol}(\begin{bmatrix} 1 & \rho \\ \rho & 1 \end{bmatrix})$.

6.2 Calcul des percentiles

```
# Percentiles sur l'axe des simulations (axis=1)
p10 = np.percentile(paths, 10, axis=1) # Bear case
p50 = np.percentile(paths, 50, axis=1) # Médiane (scénario central)
p90 = np.percentile(paths, 90, axis=1) # Bull case
```

7. STRESS TEST GÉOPOLITIQUE

7.1 Fonction stress_test()

L'idée du stress test est simple : on perturbe volontairement trois paramètres pour simuler une crise. On choque χ vers le haut, on amplifie la vol, et on ralentit le retour à la moyenne.

```
def stress_test(params, states, horizon, n_sims, dt,
               shock=0.30, vol_mult=1.5):

    kappa_s = params[0] * 0.7      #  $\kappa$  réduit → marché plus inerte en crise
    s_chi_s = params[1] * vol_mult  #  $\sigma_\chi$  amplifiée → plus de volatilité

    # Choc initial :  $\chi$  part d'un niveau +30% au-dessus
    #  $\log(1 + 0.30)$  = choc de 30% en espace log-prix
    chi_0 = states[-1, 0] + np.log(1 + shock)
    xi_0 = states[-1, 1]  # L'équilibre fondamental n'est pas choqué
```

💡 Le choix de ne pas choquer ξ est intentionnel et économiquement justifié : les chocs géopolitiques de court terme ne modifient pas les coûts de production long terme ni la demande structurelle. Seule la composante cyclique χ est affectée.

Paramètre	Scénario Normal	Scénario Stress	Impact
κ	calibré MLE	$\kappa \times 0.7$	Demi-vie 43% plus longue
σ_χ	calibrée MLE	$\sigma_\chi \times 1.5$	Volatilité +50%
χ_0	état Kalman	état + $\ln(1.30)$	Prix initial +30%
ξ_0	état Kalman	inchangé	Fondamental préservé

8. DASHBOARDS INTERACTIFS PLOTLY

8.1 Architecture des figures

Les 6 fonctions de graphiques suivent toutes le même schéma : on crée une Figure, on ajoute les traces, on configure le layout, et on retourne l'objet. La sauvegarde est gérée séparément par `save_all_figs()`.

Patron commun de configuration du layout

```
fig.update_layout(
    template='plotly_white',      # Fond blanc — standard professionnel
    hovermode='x unified',        # Tooltip unifié sur l'axe X
    yaxis=dict(
        side='right',             # Prix à droite — convention
        title='Bloomberg/Reuters',
        showgrid=True,
        gridcolor='#F2F2F2'
    ),
    xaxis=dict(
        rangeselector=dict(       # Boutons 1a / 5a / Tout
            buttons=[...]
        )
    )
)
```

)

8.2 Détail des 6 dashboards

Dashboard	Type Plotly	Spécificité technique
01 - Candlestick	go.Candlestick + go.Bar	make_subplots 2 lignes, shared_xaxes
02 - Décomposition	go.Scatter × 3	customdata pour basis_pct dans tooltip
03 - Prévvision	go.Scatter + fill toself	Polygone d'intervalle de confiance
04 - Risque	make_subplots + go.Histogram	Dual axes, vline pour VaR
05 - Stress	go.Scatter + fill tonexty	Remplissage entre courbes normal/stress
06 Backtest	go.Scatter + yaxis2	Axe Y secondaire pour les résidus

9. EXPORT ET ORCHESTRATION FINALE

9.1 Export Power BI (export_powerbi())

Les CSV utilisent le point-virgule comme séparateur et la virgule comme décimale — format attendu par Excel et Power BI en mode français.

```
df_ts.to_csv('PowerBI_Oil_TimeSeries.csv',
            index=False,
            sep=';',      # Séparateur européen
            decimal=','   # Décimale européenne
)
```

9.2 Fonction main () Orchestration

```
def main():
    # Pipeline séquentiel – ordre d'exécution garanti
    df_base = load_and_clean(DATA_FILE)      # [1/8]
    df_piliers = build_piliers(df_base)      # [2/8]
    params = calibrate(prices_mat, ...)      # [3/8]
    states, ... = extract_states(params, ...) # [4/8]
    sim_paths = monte_carlo(params, ...)     # [5/8]
    stress_paths = stress_test(params, ...)  # [6/8]
    figs = {fig_candlestick(), ...}         # [7/8]
    save_all_figs(figs)                     # [7/8]
    export_powerbi(...)                      # [8/8]

    # Ouverture automatique du dashboard principal
    import webbrowser
    webbrowser.open(f'file://{main_html}')

if __name__ == '__main__':
    main() # Exécution uniquement si lancé directement
```

💡 Le guard 'if `__name__ == "__main__"`' est une bonne pratique Python : il permet d'importer les fonctions de ce fichier dans un autre script (tests, notebooks) sans déclencher l'exécution complète du pipeline.

9.3 Constantes globales et paramètres configurables

Constante	Valeur par défaut	Pour la modifier
DATA_FILE	Db_CL.xlsx	Nom du fichier Excel d'entrée
PILIERES_JOURS	[20, 40, 60]	Maturités en jours pour l'interpolation
DT	1/252	Pas de temps (252 jours ouvrés/an)
N_SIMS	500	Nombre de simulations Monte Carlo
HORIZON	252	Horizon de prévision en jours ouvrés
SHOCK_MAG	0.30	Amplitude du choc stress test (+30%)
VOL_MULT	1.5	Multiplicateur de volatilité en stress

Ce document couvre l'intégralité du fichier schwartz_smith_oil.py (≈ 700 lignes).